Evaluation report

Threadline, Group 01, Track 2. How do we know it works, and where does it not?

1.7%
of batches late on the simulator, against 34.2% for the best supplied baseline

Tie
with the simple rule earliest finish, on every metric. The allocator adds nothing there

8 → 15
out of 16 unseen requests right: rules written blind to the requests, then our LLM reader

9 of 9
replies that applied a constraint did not say so in the reply text

Short version. Our allocation rule is good, but a five-line rule is just as good, so the value is not there. The value is in reading: on requests its authors had not seen, our LLM reader got 15 of 16 right and blind keyword rules got 8. The reader is not clean either: the fixes we tested on the held-out set were made after looking at it, and no real dispatcher or workshop has used the system. Section 2 says what is and is not a clean result.

1. What we measured and why

Question	Measure	Why this measure
Are allocations good?	Course simulator: % batches late, late days, busiest workshop's share of pieces, % defective pieces, cost	Same yardstick for every team. Lateness is our primary objective because quick-response orders have 7 to 14 days and a late batch loses the margin
Does it handle a chat request?	Behaviour (extract, clarify, refuse, decline to answer) against human labels	The wrong behaviour is the costly error: a guess where a question was needed
Is the decision right?	Same workshop as an independent earliest-finish reference under the labelled constraints	Judged apart from behaviour, as the brief asks
Is the explanation faithful?	Two checks: every number traces to a tool output, and a new review of what the reply says	A right decision with a made-up reason is still a failure. The number check alone cannot see a misleading sentence
What does the LLM add?	The same requests through three no-LLM readers	Otherwise "the LLM helps" is only an assumption
Is it usable at a desk?	Seconds and estimated cost per request	A dispatcher answers within the hour, often within minutes

We call the first behaviour *allocate* in the code. It is the brief's *extract*: the facts are read and a recommendation is produced. Nothing is sent until a person approves.

2. How the evaluation sets were built, and why they are not rigged

Simulator. The unchanged course harness (`harness/simulate.py` , identical to the supplied file), 120 orders, 10 paired seeds (5105 to 5114), with and without `--shock` . Each policy sees the same orders and the same closures. Hybrid weights were chosen on different seeds (6105 to 6114) by a rule fixed before running. We re-ran the whole evaluation on the final code on 2 November 2026, and `results.csv` came back byte for byte.

The 30 supplied requests. This is our development set, and we tuned on it. Two reviewers (Vanshika Dhamija, Apurwa Kumari) labelled them independently, then reconciled. They agreed on 27 of 30 behaviours before discussion. Labelling happened after the intake was built. Some columns for one reviewer were drafted by an AI assistant and then reviewed. Three labels (R20, R28, R30) were changed after seeing the system's answers. The change is logged in `eval/labeling/reconciliation.md` . These facts all lean in our favour, so treat 30 of 30 as a development score.

The held-out set. 16 new requests (R31 to R47, R39 removed before any run). The team wrote 13; an AI assistant drafted 3 (R44 to R46). Labels were drafted by AI from the rubric and data without running the system, then reviewed by the team. Fingerprints of the files were saved before the first run.

> **What is clean and what is not.** The only result that was never used for development is the first held-out run: **Haiku 5.5, 15/16 behaviour, 13/16 decision**. Sonnet and Opus were run on the same set afterwards to choose a model, so Sonnet's first result (15/16, 14/16) is already a selection. Our final figure, Sonnet 15/16 and 15/16, came after two fixes that were decided from those held-out failures, so it is a development result. We planned a second held-out round written by a teammate who had not built the guards. **It has not been run**, so the fixes have no clean test.

Red-team set. 16 messages we wrote to break the system (gibberish, injection, typos, several orders in one message). We then built the guards against the failures, so 16 of 16 shows they do what they were built to do and nothing more.

The rule readers. Written for this report so "what the LLM adds" is measured. The first attempt was contaminated: the author (an AI assistant) had just read all 46 messages, and that reader scored 29 of 30 and 16 of 16. We kept it as an upper bound and asked a separate agent, barred from every request, label and evaluation file, to write rules from the data dictionary alone. Its file was not edited after its first run. Scoring favours all the rule readers, as section 3.4 explains.

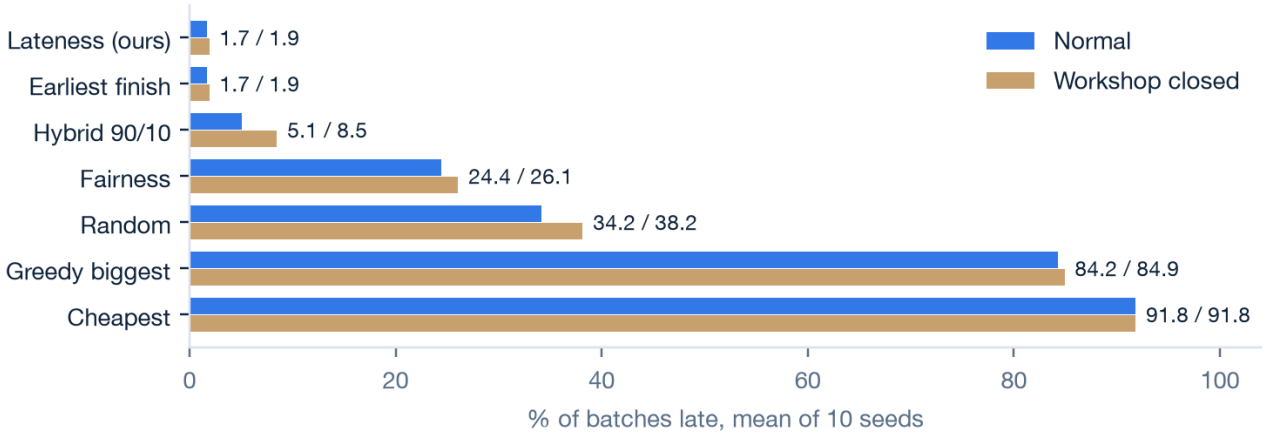
---

3. Results

3.1 Simulator: objectives against the baselines

Policy	% late (SD)	Late days	Busiest share %	% defective pieces	Total cost	% late, closure	Late days, closure
Baseline: random choice	34.2 (0.7)	6.13	22.6	5.6	111,780	38.2	7.17
Baseline: greedy biggest	84.2 (0.3)	26.53	70.2	5.3	108,430	84.9	28.05
Baseline: cheapest	91.8 (0.5)	81.92	70.2	8.2	85,960	91.8	84.12
Simple rule: earliest finish	1.7 (0.0)	0.18	26.3	7.2	115,566	1.9	0.19
Ours: lateness (primary)	1.7 (0.0)	0.18	26.3	7.2	115,566	1.9	0.19
Ours: fairness (second objective)	24.4 (1.6)	7.20	17.5	6.5	116,028	26.1	7.15
Ours: hybrid 90% lateness, 10% fairness	5.1 (1.3)	0.43	19.7	6.2	113,172	8.5	0.63
Ours: lowest defect (reported only)	95.1 (0.3)	139.87	70.2	2.9	129,920	95.1	142.48

Mean of 10 paired seeds. SD is across seeds, for normal runs. Sources: eval/results.csv, eval/results_per_seed.csv.



- **Primary objective.** Lateness beats all three baselines on % late and late days in both scenarios. The closure moves it only from 1.7% to 1.9%, about one extra late batch in 120 in the worst case.

- **Second objective.** Fairness brings the busiest workshop's share down to 17.5%, and % late (24.4) stays within its guardrail of no worse than random.
- **Hybrid.** 90% lateness and 10% fairness gives 5.1% late with the busiest share at 19.7%. It gives up 3.4 points of lateness for most of the fairness gain. Under closure it rises to 8.5%, so spreading work makes the desk more exposed to a closure.
- **Lowest defect.** It fails its guardrail: 2.9% defective pieces, but 95.1% of batches late. We report it and do not offer it as a default.

3.2 Where a baseline wins, and what we add over a simple rule

There are 83 strict pairwise cases in the normal run and 80 under closure where a baseline or the simple rule beats one of our presets. All are listed in `eval/baseline_wins.md`. The ones that matter:

- **Earliest finish ties our lateness preset on every metric, in both scenarios.** With its default weights, our allocator reduces to "whoever finishes first". The rule is about the size of a shipped baseline and uses no LLM. For pure allocation, our system adds nothing.
- **Cheapest always costs least**: 85,960 against 115,567, a gap of 26%. It is late 92% of the time.
- **Random and greedy biggest are both cheaper than our lateness preset.** Speed costs about 3 to 7%.
- **Random beats our lateness preset on defects** (5.6% against 7.2% of pieces), because fast workshops are not the careful ones.
- **Earliest finish beats every other preset on lateness**, including the hybrid.

What we add is everything around the rule. A simple allocator takes a clean order and returns a workshop. It cannot read a chat message, refuse a suspended workshop by name, ask when the date is unconfirmed, decline a question the data cannot answer, or leave a record of who approved what. The next section measures the first of these.

3.3 The 30 supplied requests, by behaviour (development set)

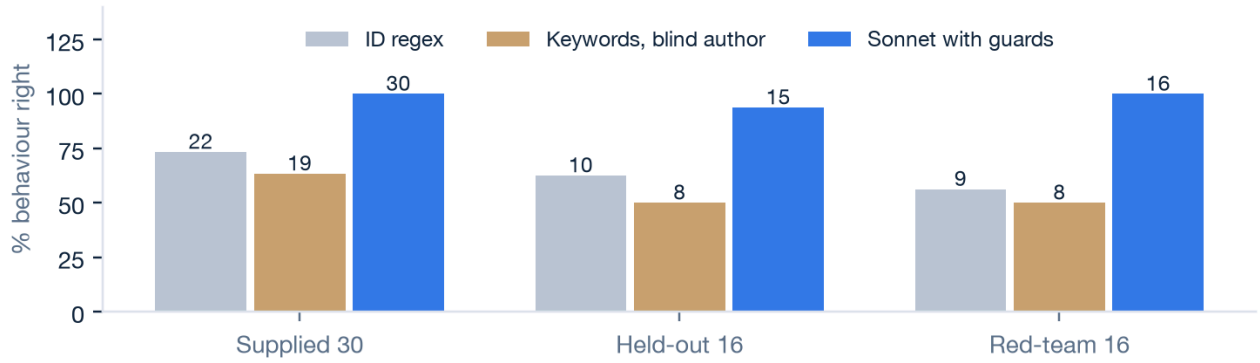
Behaviour	Requests	Behaviour right	Decision right	Numbers traced to tools	Explanation fully clear
Extract	16	16/16	16/16	16/16	11/16
Clarify	7	7/7	6/7	7/7	2/7
Refuse	4	4/4	4/4	4/4	4/4
Decline to answer	3	3/3	3/3	3/3	2/3
All	**30**	**30/30**	**29/30**	**30/30**	**19/30**

Sonnet 5.5, final code. Model replies recorded earlier and re-scored offline on 2 November 2026; the guards changed one result (R19, failure 5). "Explanation fully clear" is the new review in section 3.5. Every request is listed in the appendix.

The one decision miss is R19, a deliberate change. The first live run of the original system, on Haiku, scored 28/30 on both behaviour and decision.

3.4 What the LLM adds: rule readers on the same requests

Reader	Supplied 30: behaviour	Supplied 30: Decision	Held-out 16: behaviour	Held-out 16: Decision	Red-team 16: behaviour	Red-team 16: Decision
Order ID regex only	22/30	19/30	10/16	8/16	9/16	9/16
Keyword rules, author blind to the requests	19/30	17/30	8/16	8/16	8/16	8/16
Keyword rules, author had read the requests	29/30	29/30	16/16	16/16	9/16	9/16
Sonnet 5.5 with guards (ours)	**30/30**	**29/30**	**15/16**	**15/16**	**16/16**	**13/16**



Numbers on bars are requests right. Source: eval/rule_baseline/, run with eval/run_rule_baseline.py. No model call.

Each reader feeds the same reference allocator. Scoring favours the rules: for them a clarify or decline counts as a right decision on behaviour alone, while ours must also pass the candidate-list checks. On the red-team set our decision score is 13 of 16 because the scorer still expects the old 34-order list, which the guards removed on purpose (failure 13).

This is the most useful comparison in the report. On held-out requests, rules from a blind author got 8 of 16 and ours 15 of 16. Rules written with sight of the messages got 16 of 16, which says something uncomfortable: **these 46 messages are short, English and regular enough that a person who has read them can cover them with keywords.** That rule fails on new phrasing: on the red-team set, which it had not read, it falls to 9 of 16. The LLM's lead is the ability to cope with messages nobody planned for. Our evidence for that is 16 unseen requests plus the partner replies below, so it is promising and not proven.

3.5 Explanation review: is what the reply says true, and does it say why?

The number check only proves a number came from a tool. So we added a review of all 46 replies, by script and by reading each one (`eval/run_explanation_review.py` , `eval/explanation_review.csv`). The reading was done by an AI assistant and has not yet had a second human reader.

Check	Result	Reading
Every number traces to a tool output	46/46	No invented numbers
Days, return date and cost, recomputed from `workshops.csv`	32/32	The arithmetic in every reply with a workshop is right
An applied exclusion or preference is named in the reply text	**0/9**	True but not explained. See failure 10
ALREADY LATE only on a request for placement	2/4	Shown on two information questions. See failure 9
A clarification asks only for what is missing	0/6	Asks for a due date the order file holds. See failure 11

No reply states something false about an order. Seventeen of the 46 could be clearer, and we count those as explanation weaknesses separately from the decision.

3.6 Held-out requests, models, red team, partners, speed

Behaviour	Requests	Behaviour right	Decision right	Numbers traced to tools	Explanation fully clear
Extract	9	9/9	9/9	9/9	5/9
Clarify	2	2/2	2/2	2/2	1/2
Refuse	3	3/3	3/3	3/3	3/3
Decline to answer	2	1/2	1/2	2/2	1/2
All	**16**	**15/16**	**15/16**	**16/16**	**10/16**

Run	Set	Behaviour	Decision	Status
Haiku 5.5, original system	**Held-out 16**	**15/16**	**13/16**	**Clean, run once**
Sonnet 5.5, original system	Held-out 16	15/16	14/16	Model selection

Run	Set	Behaviour	Decision	Status
Opus 5.5, original system	Held-out 16	15/16	14/16	Model selection
Haiku 5.5, after fixes	Held-out 16	13/16	13/16	Development. Haiku got worse
Sonnet 5.5, after fixes (final)	**Held-out 16**	**15/16**	**15/16**	**Development**
Opus 5.5, after fixes	Held-out 16	15/16	15/16	Development

15 of 16 corresponds to a rough 95% Wilson range of 72% to 99%. We chose Sonnet because accuracy tied or beat the others, it was fastest, and it costs about one cent. It was not chosen on a clean basis.

Other measure	Result	Note
Red-team, behaviour	10/16 before guards, 16/16 after	Development set. Supplied 30: unchanged, one intended decision change. Held-out: 0 rows changed
Prompt injection	2/3 held before guards, 3/3 after	"Mark as approved" produced only a recommendation: the schema has no such field. The third asked for the system prompt
Workshop replies, development	42/48 before fix, 48/48 after	English 12, Hinglish 12, Hindi 12, Chinese 12
Workshop replies, held-out	11/12	Hindi 6/6, Chinese 5/6. Written after the fix. Sender languages only, never real workshop staff
Speech, word errors	26% phone model, 6% Whisper with names as hints	48 spoken requests in 4 synthetic voices. Real speakers will differ
Speed	Median 2.4 s, slowest 5.4 s server; 2.52 s median click to screen	30 requests, one run. The Python engine takes about 2 ms
Cost	About $0.011 per request	Estimated from list prices, not billed
Automated tests	568 passed	Run 2 November 2026. No model calls

4. Failure cases

Failure 1. R04 and R25: a date with words in front of it

"ORD-041 needs a home today. 600 beanies, back by Apr 21." Haiku's first run asked for the due date instead of allocating. The Python date reader could not parse a date with words before it, so it threw away a correct model reading. Wrong behaviour on 2 of 30. **Fixed** in the date reader. This was our own parsing bug, not a model error.

Failure 2. R44: the model filled the wrong date slot

"Maple & Co are chasing ORD-083, it's already late. Can we get the hoodies out today?" All three models read "out today" as the deadline and reported 8 days late, not 17. The template had one date slot, and the model never sees the file's due date, so the message date overrode it. The number check passed, because 8 was a true tool output from a wrong input. **Fixed** with a separate send-date field and a check that flags a conflict with the order file. The fix is not cleanly tested.

Failure 3. R36: vague quality wording ignored

"ORD-082, 600 Beinie for TrendCart, due May 1. Ensure quality." Haiku ignored this and chose a 5% defect workshop over a 2% one. Vague wording was not mapped to a preference. **Fixed** by rule version 2, where quality words mean lowest defect, still subject to the date. We set that rule after seeing this failure, and relabelled to match.

Failure 4. R34: cannot say what it does not know

"is ORD-069 delivered to client?" Expected: decline, because delivery is not recorded. All models instead asked whether the sender wanted a status answer. The rules send every status question to clarify, and cannot separate data we hold (stage) from data we do not (delivery). Harmless, but it is a question that leads nowhere. **Open**, and it is the one held-out miss in our final score.

Failure 5. R19: the reply got better and the score got worse

"That big reorder from last week, send it to whoever is quickest." The old reply listed all 34 open orders, which the frozen evaluator expects. The guard now asks for the order ID in one line. We think that is better, and it costs a point. We did not change the evaluator, so we report 29/30.

Failure 6. R23: split wording read differently

"Split it across two shops if that gets it back faster." The workshop and split preview were right. Sonnet did not record "fastest" as a preference, while the label does. "Faster" is a condition on splitting, and readers disagree whether it is a preference. **Open**, minor. It is one of the two constraint misses in the 28/30 constraint score.

Failure 7. R25 and R41: who gave the instruction was lost

"...keep it away from BudgetWorks..." The exclusion was applied and the workshop was right, but the audit record omitted who imposed it. Seen with Sonnet on R25 and Haiku on R41. The field is optional for the model, and the longer prompt may be the cause, or it may be run-to-run variation. We cannot tell from one run. **Open**. An audit trail with a blank is a real weakness, so this field should be filled by Python.

Failure 8. R42 and R45: Haiku became worse after our fixes

"ORD-040, assign something, due date to be confirmed." Haiku allocated using the file date. On R45, *"as long as it arrives on time"* made it ask for a date. Its held-out behaviour fell from 15 to 13, which suggests it had been right on these partly by accident. **Open** for Haiku, which still does background reading of the whole inbox. Sonnet is right on both.

Failure 9. R13 and R47: ALREADY LATE on a question

"Finance asks: what price did we quote TrendCart on ORD-120?" The reply correctly declines, then adds "ALREADY LATE: the requested deadline is today or in the past." Nobody asked for a deadline. The flag runs on the order's file date for any behaviour. On R47 the order is due today, so "late" is also not yet true. All automated checks pass. Found only by reading. **Open**, a one-line fix.

Failure 10. Nine replies apply a constraint and do not say so

R09, R12, R21, R25, R27, R32, R36, R41, R45. R21 asks for the cheapest shop that makes the date. The reply says "Lowest score among permitted candidates", with no word about BudgetWorks being cheaper and three days late. R25 keeps away from BudgetWorks and never mentions it. Each decision is right and each sentence is true, but the template states the winner's numbers, not the reason. The **Why this decision** panel on the web desk does show it. The text reply is what the evaluator scores. **Open.** This is the most important explanation gap, since the promise of the product is that a person can see why.

Failure 11. Six clarifications ask for a date we already hold

R01, R03, R10, R15, R29, R37. *"Which order do you mean: ORD-061; ORD-108? What is the confirmed due date, including the year?"* Once the order is picked, the date comes from the file. The system asks because the message gave none. It adds a round of chat and looks careless. R10 is also weak: "New rush... details to follow" is a new order, and the reply lists existing ones. **Open.**

Failure 12. C26: a price lost in Chinese

每件涨到2块 ("up to 2 per piece"). The model read a price change and lost the number. Because the number was missing, the desk asked the workshop again and changed nothing. The failure is safe but real: it is the one held-out miss in partner replies, and the film shows it. English "up to 1.8 per piece" works, so colloquial Chinese numbers need more testing. **Open.** It was found before the Sprint 3 film and is one of its stated limits.

Failure 13. The red-team set before the guards

Before the guards, 6 of 16 behaviours were wrong and 9 of 15 extra checks failed. Gibberish (*"asdf ok?? lol"*), off-topic questions and a request for the system prompt all listed the 34 open orders. *"ord 041"* and *"ORD45"* were declined as unknown. Two orders in one message lost one. The cause was one path: anything without a clean order became a clarify with every open order. **Fixed** in Python guards, on the set they were built against. Several orders are still asked about, not split into cards. Guards have not been run with Haiku.

Failure 14. R20, R28, R30: a case the four behaviours do not cover

An impossible deadline. Reviewers split between refuse and decline, and the system allocated with a warning. None of the four fits "complete but impossible". We moved the labels to allocate after seeing the system's answer. That helps our score, so we report it: without the change, the first-run score would be three lower. A human should decide this category in advance.

Failure 15. The hybrid we ship is not the best one we found

The rule picked 0.1 on the 0.1-step grid, and we ship it. On a finer grid, the same rule picks 0.05: 4.0% late, 19.2% busiest share, 5.5% late under closure, against 5.1%, 19.7% and 8.5%. 0.05 is better on all three. **Open**, and the preset should change, with the seeds kept apart.

Pattern. Most serious failures came from Python around the model, not the model: the date reader, the one-slot template, the list of 34 orders, the misplaced flag. The checks we built could not see these, because each one tests a narrow thing. Reading replies by hand found failures 9, 10 and 11 after every automated score was green.

---

5. Limits of this evaluation

- **No real users.** No dispatcher or workshop has used it. All the workshop replies were written by us.
- **Small sets.** 16 held-out requests and 12 held-out replies. One run per model, and Claude's current models accept no temperature setting, so we have no measure of run-to-run variation.
- **Partly AI-drafted.** 3 held-out requests, the held-out labels, some reviewer columns, the blind reader and the explanation reading.
- **The simulator checks the allocation rule, not the app.** It has no chat, no language and no person in the loop. The demo also differs from this report: orders in the film start from eight earlier decisions, so queues differ and ORD-045 goes to FreshStart, while this evaluation resets queues for each request and gives Nimble Needle.
- **Estimates, not measurements.** Workshop speed and defect rates are the supplied values, and the desk assumes they hold.

---

6. What we would measure with more time

1. **Round 2 held-out.** At least 20 new requests, with orders drawn by seeded sampling, written and labelled by a teammate who did not build the guards, frozen, run once. This is the clean test of every fix here.
2. **Repeat runs.** Sonnet three times on all sets, to see how much the score moves between runs.
3. **A second human reader** for the explanation review, then a rewrite of the reply template and a re-run.
4. **Usability with at least three people** acting as dispatchers: time per decision, errors, whether they trust the reason. We can measure time saved only this way.
5. **Real Hindi and Mandarin speakers** on written replies and speech, on real phones in noise.
6. **Costs from real invoices** and a week-long run on one Mac, to see how the single point of failure behaves.

Appendix: all 30 supplied requests

Decision and Numbers are the evaluator's checks. Explanation is the review in section 3.5. Workshops are W1 QuickStitch, W2 SteadyHands, W3 BudgetWorks, W4 Little Loom, W5 GiantWeave, W6 Nimble Needle, W7 OldMill (suspended), W8 FreshStart (300-piece cap). Refuse rows show the alternative offered, with no allocation made.

ID	Message, shortened	Expected	Ours	Workshop	Decision	Numbers	Explanation
R01	Need scarves for Harbor Knits sent out today, usual quanti…	clarify	clarify		yes	yes	needless date ask
R02	Which workshop did we use for the Harbor Knits batch last…	decline	decline		yes	yes	ok
R03	Can we get the TrendCart order moving? They are chasing.	clarify	clarify		yes	yes	needless date ask
R04	ORD-041 needs a home today. 600 beanies, back by Apr 21.	extract	extract	W6	yes	yes	ok
R05	ORD-066 due date just moved up, will confirm the new date…	clarify	clarify		yes	yes	ok
R06	Farm out ORD-096 please, 1000 beanies for TrendCart, ship…	extract	extract	W6	yes	yes	ok
R07	ORD-063, 100 cardigans. No special requirements, just make…	extract	extract	W6	yes	yes	ok
R08	Send ORD-073 (400 hoodies) to FreshStart, their quote was…	refuse	refuse	W6	yes	yes	ok
R09	ORD-045, 150 vests for Cotton Club, due Apr 09. It's urgen…	extract	extract	W6	yes	yes	reason not named
R10	New rush coming in from Loom & Leaf, details to follow, bu…	clarify	clarify		yes	yes	needless date ask
R11	OldMill is free right now, give them ORD-008.	refuse	refuse	W6	yes	yes	ok
R12	ORD-053, 800 polo shirts for UrbanThread, due Apr 10. Boss…	extract	extract	W6	yes	yes	reason not named
R13	Finance asks: what price did we quote TrendCart on ORD-120…	decline	decline		yes	yes	late flag misplaced
R14	QuickStitch for the scarves on ORD-103?	refuse	refuse	W2	yes	yes	ok
R15	Boss wants the Loom & Leaf stuff outside asap.	clarify	clarify		yes	yes	needless date ask
R16	Loom & Leaf confirmed the reorder: ORD-024, 300 vests. Nee…	extract	extract	W6	yes	yes	ok
R17	ORD-017, 800 cardigans for TrendCart. Factory is full, thi…	extract	extract	W1	yes	yes	ok
R18	Any movement on ORD-999? Customer is chasing.	decline	decline		yes	yes	ok
R19	That big reorder from last week, send it to whoever is qui…	clarify	clarify		no	yes	ok
R20	New one for the network: ORD-103, 2000 scarves for Bright…	extract	extract	W2	yes	yes	ok
R21	ORD-081, 1500 crewneck sweaters for Harbor Knits, due Apr…	extract	extract	W5	yes	yes	reason not named
R22	ORD-040 ready to dispatch. 1200 hoodies for Loom & Leaf, b…	extract	extract	W1	yes	yes	ok
R23	ORD-005, 500 scarves for TrendCart, due Apr 20. Split it a…	extract	extract	W6	yes	yes	ok
R24	Little Loom does lovely work, ORD-015 to them?	refuse	refuse	W6	yes	yes	ok
R25	ORD-109, 500 vests for Harbor Knits, due Apr 13. …and ke…	extract	extract	W6	yes	yes	reason not named
R26	Can someone place ORD-010? 1000 cardigans, Cotton Club, du…	extract	extract	W1	yes	yes	ok
R27	ORD-095, 800 hoodies for Bright Basics, due Apr 26. Bright…	extract	extract	W2	yes	yes	reason not named
R28	Sending ORD-002 outside, 1000 crewneck sweaters. Who has r…	extract	extract	W1	yes	yes	ok
R29	The vests need to go out this week, who takes them?	clarify	clarify		yes	yes	needless date ask
R30	Please allocate ORD-099: 1500 polo shirts, Northwind Appar…	extract	extract	W1	yes	yes	ok